

User Documentation Index

This directory contains user-facing guides and documentation for the Hekat DSL. ---

Quick Start

New to Hekat? Start here: 1. [../QUICKSTART.md](#) (5 min) - 5-minute introduction ★ 2. [../examples/README.md](#) (10 min) - Hands-on examples with full layering 3. [DSL-SYMBOLIC-VISUAL-GUIDE.md](#) (5 min) - Quick symbol reference 4. [DSL-COMPLEXITY-LEVELS.md](#) (15 min) - Understand the 7 levels **Prefer hands-on learning?** → [../examples/level6-examples-probabilistic.md](#) - Probabilistic workflows → [../examples/level6-examples-error-handling.md](#) - Error handling patterns **Need voice/accessibility?** → [DSL-VERBAL-INTERFACE.md](#) - Complete voice guide **Want comprehensive coverage?** → [DSL-ORCHESTRATION-COMPREHENSIVE.md](#) - 150-page deep dive ---

Documents

[DSL-SYMBOLIC-VISUAL-GUIDE.md](#)

Quick Reference | 5 minutes Concise visual reference:

• Agent	→ Sequential
● Skill	Parallel
■ Command	+ Combination
○ Workflow	: Specification

Includes: - Core symbols and operators - Visual patterns for all 6 levels - Quick pattern lookup table - Common composition examples - Execution model diagram **Perfect for:** Desk reference, cheat sheet, quick lookup ---

DSL-COMPLEXITY-LEVELS.md

6-Level Hierarchy | 15 minutes Progressive complexity guide: **Level 1:** Basic invocation

```
api-architect : "design REST API"
```

5-15K tokens, 2-5 min **Level 2:** Binary operations

```
research -> design  
frontend || backend
```

10-30K tokens, 5-15 min **Level 3:** Parallel streams ★

```
(/deep + /ctx7 || /orch /wflw || /meta-skill-builder) : "task"
```

40-100K tokens, 20-45 min **Level 4:** Complex orchestration

```
test -> if(pass) deploy : (fix -> retest)
```

80-150K tokens, 45-90 min **Level 5:** Workflow composition

```
workflow microservice_dev(service, domain) { ... }
```

120-250K tokens, 90-180 min **Level 6:** Meta-programming

```
fmap: (A -> B) -> Workflow<A> -> Workflow<B>
```

200K+ tokens, 3+ hours **Perfect for:** Understanding complexity, choosing the right level, planning orchestrations ---

DSL-VERBAL-INTERFACE.md

Voice Interface Guide | 30 minutes Complete voice accessibility guide: **Translation Pipeline:**

```
Natural Language → Verbal DSL → Formal DSL → Execution
```

Example:

```
SAY:      "Run deep research and context lookup together,
           then synthesize on DSL design"

VERBAL: "parallel: deep research, context lookup
        then: synthesize
        task: DSL design"

FORMAL: (/deep || /ctx7) -> synthesize : "DSL design"
```

Features: - Natural language mappings - Speakable agent names - Intent recognition patterns
- Voice commands for all 6 levels - Accessibility features - Implementation guide **Perfect for:**
Hands-free operation, accessibility needs, natural language users ---

dsl-examples.md

Practical Examples I 30 minutes Hands-on examples across all domains: - Full-stack development - DevOps and deployment - Research workflows - Data processing - Testing and QA - Documentation generation **Example - Microservice Development:**

```
research_domain("auth") ->
(
  design_api("user-service") ||
  design_database("users") ||
  design_infrastructure
) ->
implement_core ->
(unit_tests || integration_tests || api_tests) ->
deploy_staging
```

Perfect for: Learning by doing, practical patterns, real-world use cases ---

DSL-ORCHESTRATION-COMPREHENSIVE.md

Comprehensive Guide I 150 pages Complete coverage of all aspects: **Part I: Foundations** - Mathematical framework - Operator semantics - Type system **Part II: Complexity Levels** - Detailed examples for each level - Visual diagrams (80+ ASCII art) - Time and token analysis

Part III: Advanced Topics - Optimization strategies - Error handling patterns - Production case studies
Part IV: Reference - Complete operator reference - Pattern library - Mathematical laws and properties
Perfect for: Deep understanding, reference material, comprehensive learning --
-

Reading Paths

Beginner Path

- | | |
|---------------------------------|------------------|
| 1. DSL-SYMBOLIC-VISUAL-GUIDE.md | (Quick overview) |
| 2. DSL-COMPLEXITY-LEVELS.md | (Levels 1-3) |
| 3. dsl-examples.md | (Practice) |
| 4. DSL-VERBAL-INTERFACE.md | (If needed) |

Voice-First Path

- | | |
|-----------------------------|------------------------|
| 1. DSL-VERBAL-INTERFACE.md | (Voice commands) |
| 2. DSL-COMPLEXITY-LEVELS.md | (Understanding levels) |
| 3. dsl-examples.md | (Practice with voice) |

Expert Path

- | | |
|---------------------------------------|------------------|
| 1. DSL-ORCHESTRATION-COMPREHENSIVE.md | (Full deep dive) |
| 2. Review design specs (../design/) | |
| 3. Practice with Level 4-6 examples | |

Quick Reference Path

- | | |
|---------------------------------|------------------------|
| 1. DSL-SYMBOLIC-VISUAL-GUIDE.md | (Symbols and patterns) |
| 2. Keep open while working | |

Use Cases by Level

Level 1-2: Daily Development

- Single agent tasks - Simple sequential workflows - Quick prototypes

Level 3: Project Features

- Parallel research - Multi-stream processing - Feature development

Level 4: Complex Projects

- Conditional deployment - Error handling workflows - Production pipelines

Level 5: Enterprise Systems

- Reusable workflows - Multi-service orchestration - Parameterized templates

Level 6: Framework Development

- Workflow generators - DSL extensions - Meta-programming ---

Accessibility Features

The Hekat DSL is designed with accessibility in mind: ✓ **Voice-First**: Natural language interface (see DSL-VERBAL-INTERFACE.md) ✓ **Screen Reader**: Verbose mode with detailed descriptions ✓ **Progressive Disclosure**: Start simple, grow complexity ✓ **Visual Aids**: 80+ ASCII diagrams ✓ **Hands-Free**: Complete voice control ✓ **Cognitive Load**: Clear hierarchy and patterns ---

Advanced Research Documentation

FORMAL-SYMBOLIC-ENCODINGS-WORKFLOW-DSLS.md

Comprehensive Research Analysis | Academic Deep mathematical foundations for workflow DSL encodings: **Coverage:** 1. **Symbolic Calculi:** Lambda calculus, π -calculus, CCS, Petri nets 2. **Categorical String Diagrams:** Joyal-Street graphical syntax, monoidal categories 3. **Algebraic Encodings:** Lawvere theories, PROs, PROPs 4. **Compilation Strategies:** Term rewriting, DPO graph transformation, e-graphs 5. **Real-World Systems:** Apache Beam, TensorFlow, Dask **Key Theorems:** - Church-Rosser (λ -calculus confluence) - Mac Lane Coherence (monoidal categories) - Joyal-Street Soundness & Completeness - DPO graph rewriting compositionality **Includes:** 100+ formal definitions, algorithms, code examples, extensive bibliography **Perfect for:** Researchers, formal methods practitioners, compiler designers ---

FORMAL-ENCODINGS-SUMMARY.md

Visual Summary | Quick Reference Concise visual summary of formal encodings research: **Diagrams:** - Encoding taxonomy tree - Compilation pipeline flowchart - String diagram notation reference - Real-world implementation matrix **Quick Reference Tables:** - When to use which encoding - Complexity analysis comparison - Theorem and property cheatsheet - Key papers by topic **Perfect for:** Quick lookup, decision-making, teaching material ---

Related Documentation

- **Research:** [../research/INDEX.md](#) - **Design Specs:** [../design/INDEX.md](#) - **Main README:** [../README.md](#) ---

Feedback & Contributions

Found an error? Have a suggestion? Want to contribute examples? See the main [README.md](#) for contribution guidelines.